

---

---

## **Mini Project 1 – Arithmetic Logic Unit**

*Mini Project 1 – COMPENG 4DV4*

*Chris Hoftyzer 400410904*

---

---



Fixed point multiplication operations consist of a multiplication, rounding, and clipping bits [16:5] of the product. Multiplying two 12-bit numbers produces a 24-bit product. Rounding must be performed on the 24-bit product. To round, we add half the range of the discarded product bits [4:0], or decimal 16 to a positive product and 24'b11111 or decimal 15 to a negative product.

This reason for adding 15 in the negative case can be explained by rounding behaviour when the product bits [4:0] are 5'b10000, or half the possible value. In this case we want to round the absolute value up to the next integer. However, when we add 5'b10000 to 24'b10000, the result is 24'b100000, and the one is carried into position [5] of the product. In the positive case, this is correct behaviour, and the number has been rounded up. But in the negative case the number is rounded down and towards zero, so instead we add 24'd15 to ensure this case results in a round up, away from zero.

Overflow in multiplication occurs when any of the upper truncated bits don't match the result sign bit. This indicates that non redundant information has been discarded, and that an overflow has occurred.

```
//multiply
prod_long      = data_a * data_b;
prod_long_round = prod_long + (prod_long[23] ? 24'sd15 : 24'sd16);
prod           = prod_long_round[16:5];
mul_overflow   = prod_long_round[23:16] != {8{prod_long_round[16]}};
```

**Figure 3** – combinational logic for multiplication operation

MAC operations require several registers to store information from previous operations. Mac\_ff is set to high if the last instruction was part of a chain of consecutive MAC instructions. Mac\_sum\_ff stored the sum associated with the MAC chain, otherwise 12'b0, and mac\_overflow\_ff is high if an overflow has occurred at any point during the MAC chain. Each of these registers are updated on the positive edge of the clock.

```
//mac flip flops
always@(posedge i_clk or negedge i_rst_n) begin
    if (!i_rst_n) begin
        mac_ff          <= 1'b0;
        mac_sum_ff      <= 12'h0;
        mac_overflow_ff <= 1'b0;
    end else if (valid) begin
        if (inst == 3'b011) begin
            mac_ff          <= 1'b1;
            mac_sum_ff      <= mul_data;
            mac_overflow_ff <= mac_overflow;
        end else begin
            mac_ff          <= 1'b0;
            mac_sum_ff      <= 12'h0;
            mac_overflow_ff <= 1'b0;
        end
    end
end
end
```

Figure 4 – MAC registers

Mac\_active indicates if the current instruction continues a chain of MAC instructions. This is used to determine the second operand for an addition – the current product if yes, otherwise 12'b0 if no. This sum represents the output for both multiplication and MAC operations.

MAC overflows can be caused either by new overflows during multiplication or addition on the current cycle, or by an overflow on previous clock cycles of the MAC chain.

```
//mac accumulate
mac_active      = mac_ff & (inst == 3'b011);
mac_operand_ctrl = mac_active ? mac_sum_ff : 12'b0;
mul_data        = mac_operand_ctrl + prod;
mac_sum_overflow = ((mac_operand_ctrl[11] == prod[11]) & (mul_data[11] != mac_operand_ctrl[11]));
mac_overflow     = mul_overflow | mac_sum_overflow | mac_overflow_ff;
```

Figure 5 – combinational logic for MAC operation

The remaining operations are quite simple. The XNOR operation result is simply an inverted bitwise OR. The RELU operation is implemented by using the sign bit of data\_a to choose between data\_a or 12'b0 as the output. The mean of the two inputs can be found by clipping the upper 12 bits of the 13-bit sum of the inputs. To find the absolute max of the two inputs, we first convert any negative inputs to their two's complement representation. Then we can select the larger of the two as the output result using a multiplexer. None of these operations require overflow calculations.

```
//xnor
xnor_data      = ~(data_a ^ data_b);

//relu
relu_data      = data_a[11] ? 12'd0 : data_a;

//mean
mean_long      = $signed({data_a[11], data_a}) + $signed({data_b[11], data_b});
mean_data      = mean_long[12:1];

//absolute max
abs_data_a     = data_a[11] ? (~data_a + 12'd1) : data_a;
abs_data_b     = data_b[11] ? (~data_b + 12'd1) : data_b;

abs_max_data    = (abs_data_a > abs_data_b) ? abs_data_a : abs_data_b;
```

**Figure 6** – combinational logic for xnor, relu, mean and absolute max operations

When results for all operations have been calculated, the output is assigned based on the input operation.

```
//default
o_data_w       = 12'd0;
o_overflow_w   = 1'b0;
o_valid_w      = valid;

case (inst)
  3'b000, 3'b001: begin
    o_data_w = add_sub_data;
    o_overflow_w = add_sub_overflow;
  end
  3'b010: begin
    o_data_w = mul_data;
    o_overflow_w = mul_overflow;
  end
  3'b011: begin
    o_data_w = mul_data;
    o_overflow_w = mac_overflow;
  end
  3'b100: begin
    o_data_w = xnor_data;
    o_overflow_w = 1'b0;
  end
  3'b101: begin
    o_data_w = relu_data;
    o_overflow_w = 1'b0;
  end
  3'b110: begin
    o_data_w = mean_data;
    o_overflow_w = 1'b0;
  end
  3'b111: begin
    o_data_w = abs_max_data;
    o_overflow_w = 1'b0;
  end
  default: begin
    o_data_w = 12'h0;
    o_overflow_w = 1'b0;
  end
endcase
```

**Figure 7** – output selection

Outputs are registered and updated on the rising edge of the clock.

```
//output flip flops - positive clock edge
always@(posedge i_clk or negedge i_rst_n) begin
    if(!i_rst_n) begin
        o_data_r          <= 0;
        o_overflow_r      <= 0;
        o_valid_r         <= 0;
    end else begin
        o_data_r          <= o_data_w;
        o_overflow_r      <= o_overflow_w;
        o_valid_r         <= o_valid_w;
    end
end
```

Figure 8 – output registers

The design was tested using testbenches for I0 to I8 and passed all testcases.

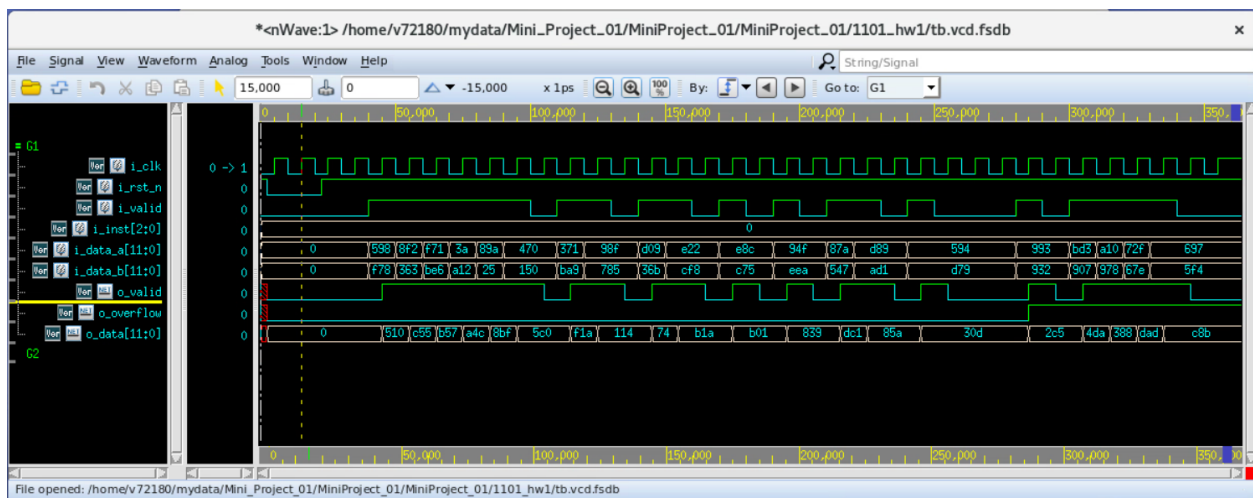


Figure 9 – Waveform for testbench I0

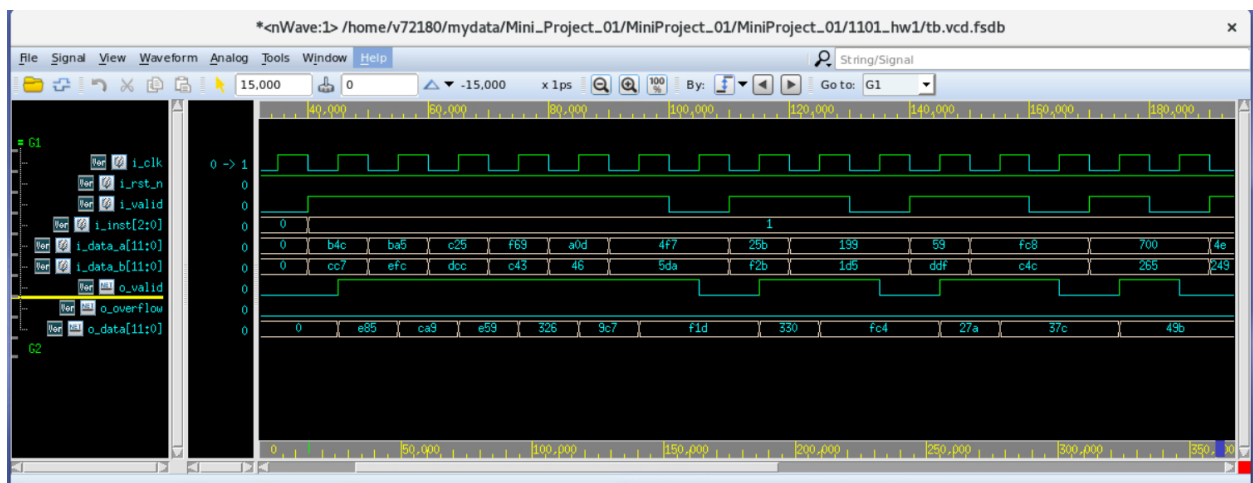


Figure 10 – Waveform for testbench I1

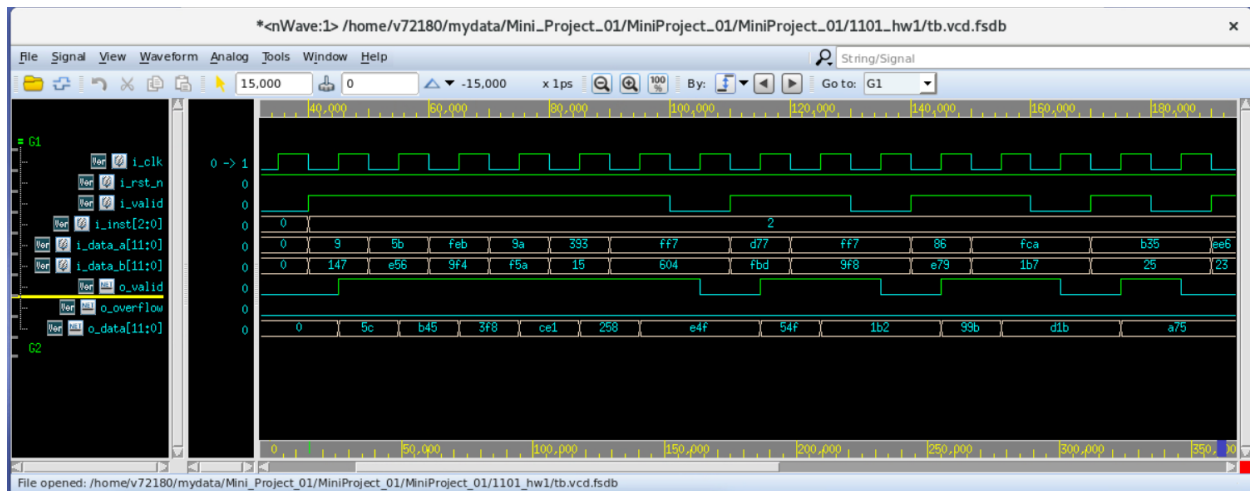


Figure 11 – Waveform for testbench I2

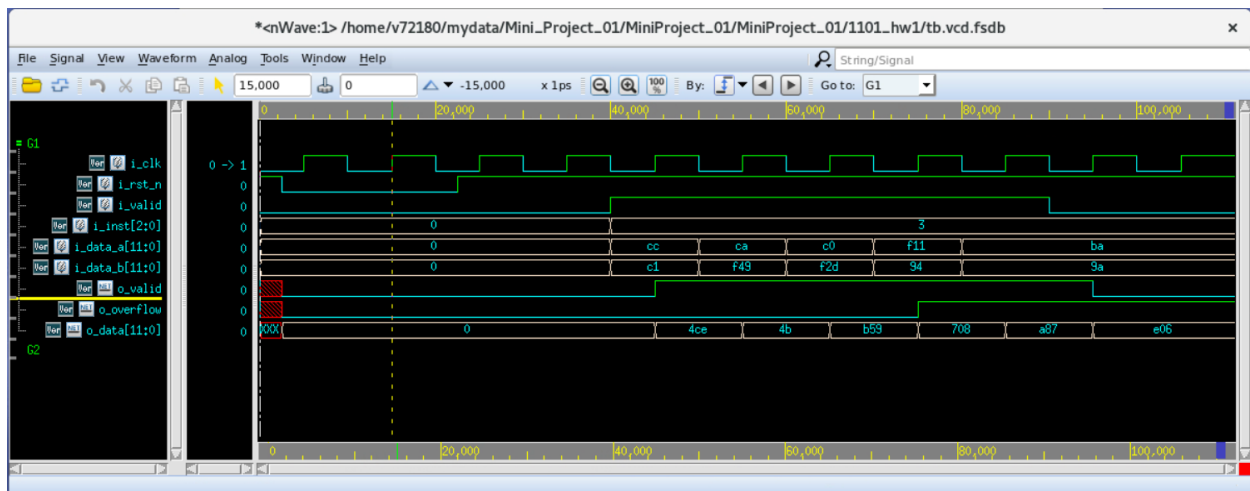


Figure 12 – Waveform for testbench I3

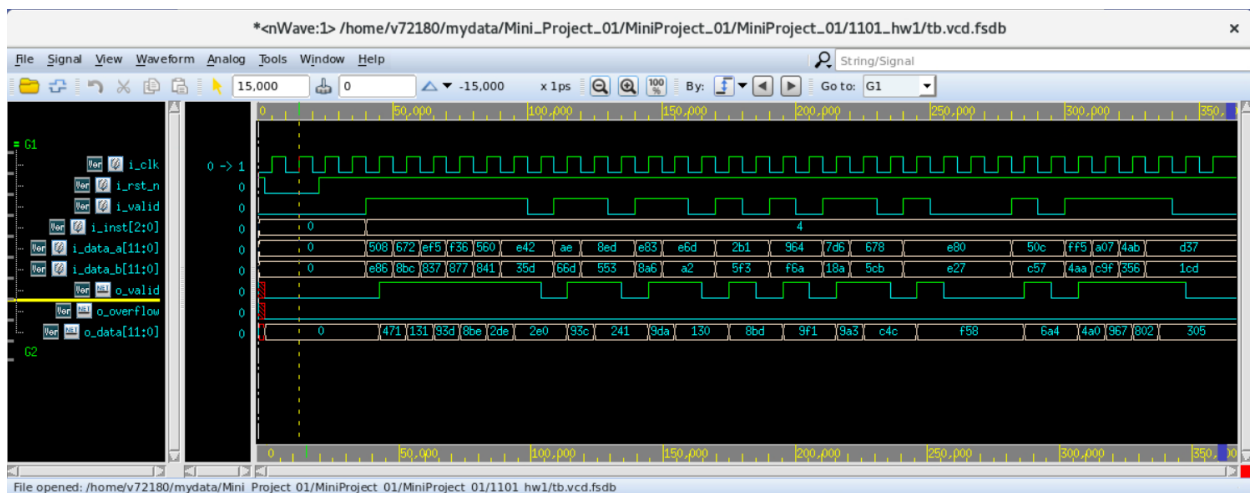


Figure 13 – Waveform for testbench I4

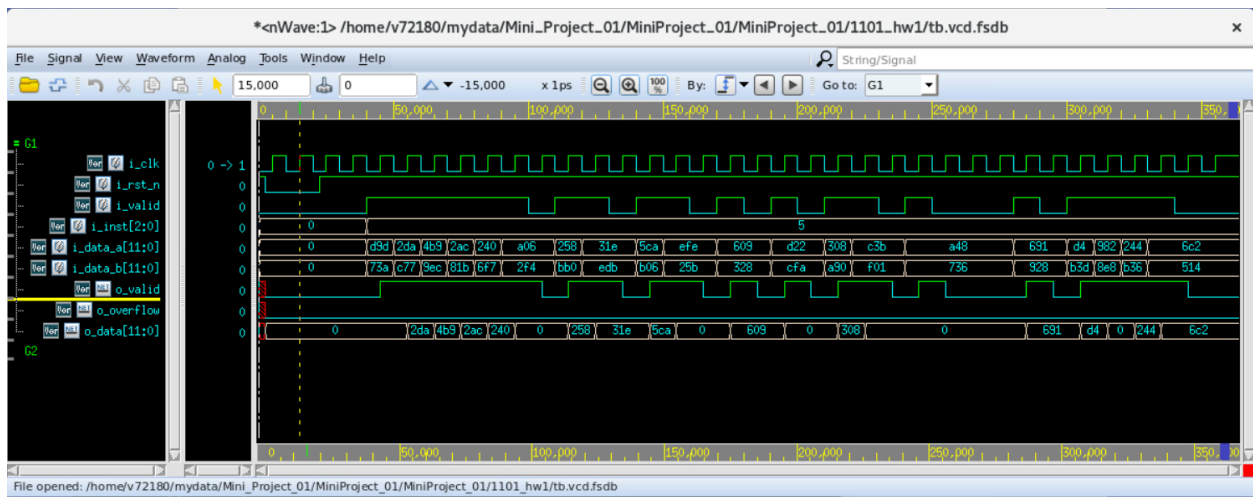


Figure 14 – Waveform for testbench I5

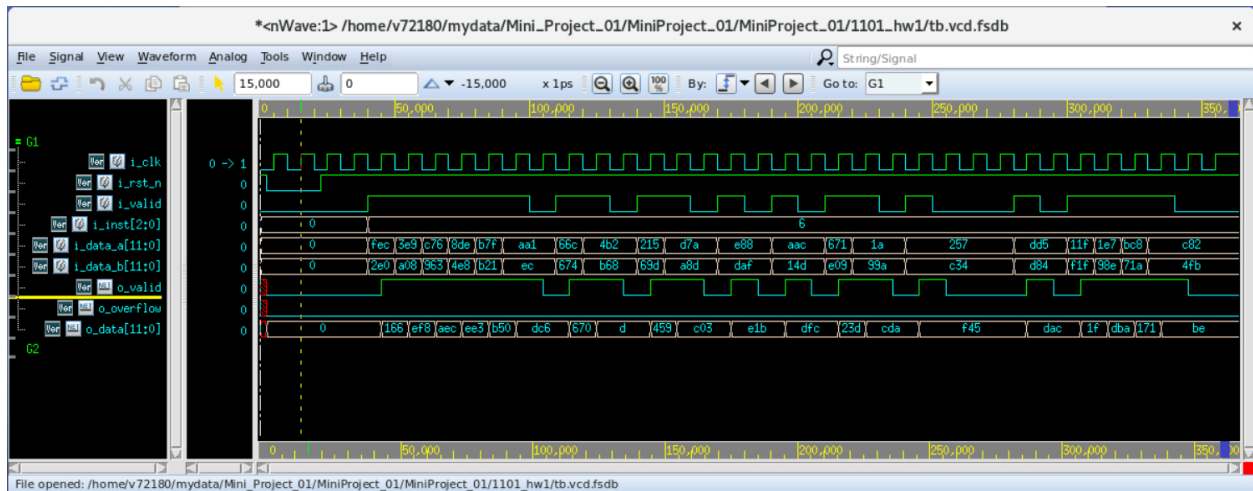


Figure 15 – Waveform for testbench I6

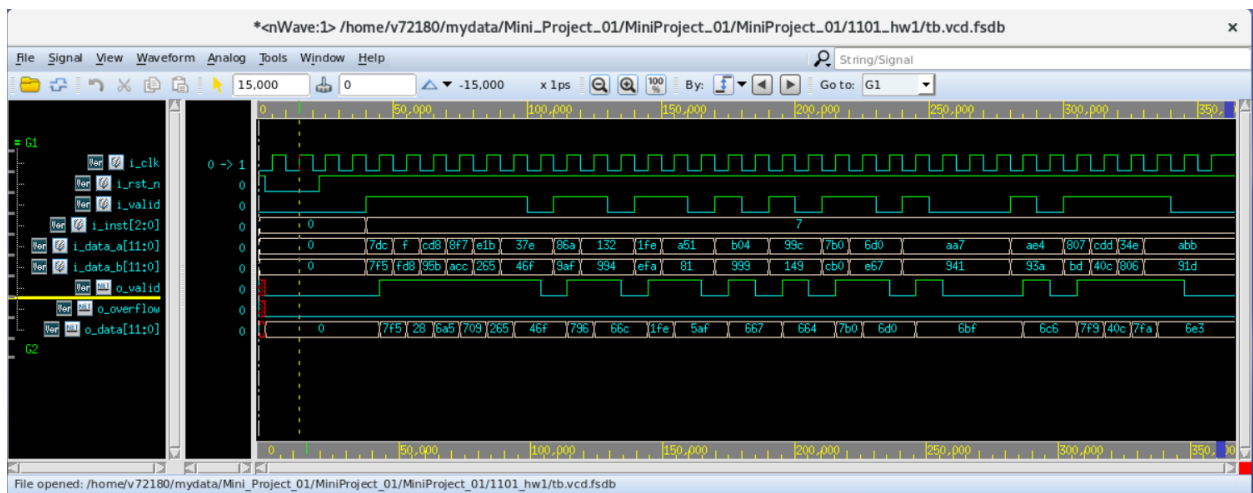


Figure 16 – Waveform for testbench I7

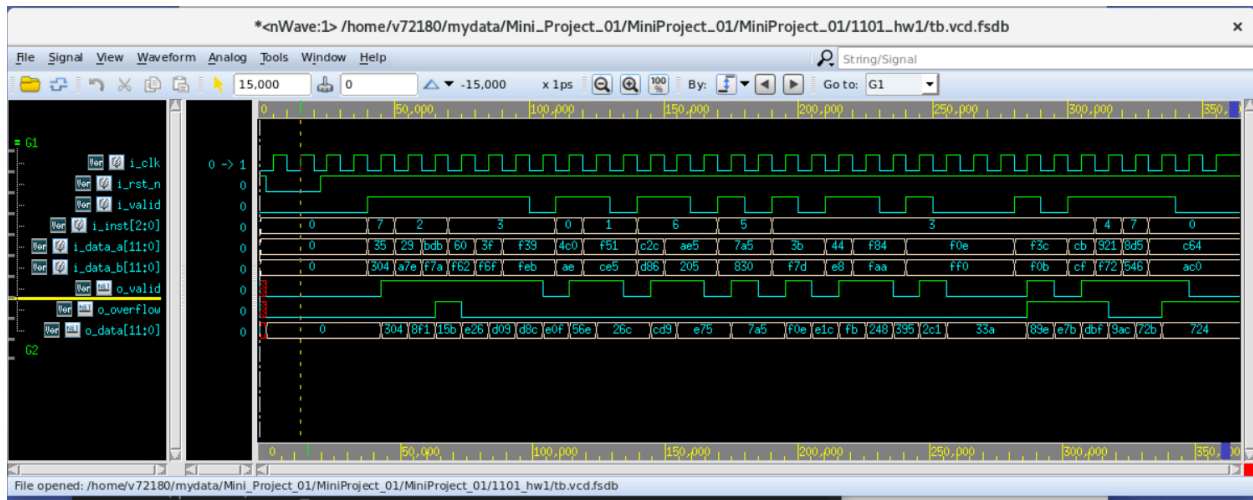


Figure 17 – Waveform for testbench I8